

Phase 8 - Report

- [PHASE 8 — GEO-SPATIAL & LOCATION INTELLIGENCE](#)
 - [Real Estate AI Decision Intelligence Platform — Phase Completion Report](#)
 - [1. PHASE STATUS](#)
 - [2. OBJECTIVE](#)
 - [3. INPUTS](#)
 - [4. WHAT WAS BUILT](#)
 - [5. SECTION 8.7 IN PRACTICE — WHAT WAS NOT INVENTED](#)
 - [6. A REAL NORMALIZATION BUG FOUND AND FIXED DURING BUILD](#)
 - [7. FULL-PORTFOLIO RESULTS \(n=90 localities — all of them, no sampling\)](#)
 - [8. TESTS EXECUTED / PASSED / FAILED](#)
 - [9. MASTER PROMPT REQUIREMENTS COVERAGE](#)
 - [10. BUSINESS VALUE](#)
 - [11. KNOWN LIMITATIONS \(see also `docs/limitations.md`\)](#)
 - [12. DEPENDENCIES](#)
 - [13. SECURITY CONSIDERATIONS](#)
 - [14. PERFORMANCE CONSIDERATIONS](#)
 - [15. PRODUCTION READINESS](#)
 - [16. NEXT PHASE DEPENDENCIES](#)
 - [17. FINAL STATUS](#)

PHASE 8 — GEO-SPATIAL & LOCATION INTELLIGENCE

Real Estate AI Decision Intelligence Platform — Phase Completion Report

Follows the Master A-Z Prompt's 10-phase architecture, Section 19 (Phase Completion Report). Matches the established convention: live PostgreSQL via `db/connection.py`, `src/<topic>/*.py` modules, orchestrators for Markdown report / numbered CSV export / charts, and an automated test suite. Per the architecture diagram, Phase 8 runs in parallel with Phases 6/7 (all three depend only on Phase 1/5, not on each other) and integrates with them in Phase 9.

1. PHASE STATUS

PHASE 8 — COMPLETE. Locality intelligence, accessibility, infrastructure intelligence, neighborhood quality, geo risk, and a documented Location Score composite are built and wired into three orchestrators. **26/26 automated tests passing.** Unlike Phases 6/7 (8,000 properties, sampled), Phase 8 operates at the **locality level — only 90 rows — so every result covers the full portfolio, no sampling.**

2. OBJECTIVE

Evaluate the location/neighborhood dimension of a real-estate opportunity (§8.1): locality growth/demand/supply, accessibility, infrastructure, neighborhood quality, and geo risk — using only real Phase 1 data, never inventing a score this platform has no basis for.

3. INPUTS

`core.localities` (90 rows: coordinates, population density, income, and 5 pre-validated 0-100 scores —

connectivity, safety, infrastructure, development, commercial), `core.cities` (6 rows: Mumbai, Bengaluru, Pune, Hyderabad, Delhi NCR, Chennai), `core.infrastructure` (209 rows: Highway/Metro/Airport/School/Hospital/Mall/Business District, each with real distance-from-center and `impact_score`), `core.market_monthly` (locality-level demand/supply/absorption history, same table Phase 6 uses).

4. WHAT WAS BUILT

Module	Master Prompt Section	Purpose
<code>data_loader.py</code>	8.2	Sources every OBSERVED input from Phase 1 tables. DB path (production) + CSV path (dev/test).
<code>accessibility.py</code>	8.4, 8.5, 8.6	Inverse-distance-weighted accessibility/infrastructure sub-scores from real infrastructure distances; neighborhood-quality assembly from OBSERVED demographic scores.
<code>geo_risk.py</code>	8.7	<code>flood_risk</code> / <code>environmental_risk</code> correctly INSUFFICIENT DATA (no such data exists anywhere in this platform); infrastructure delivery risk and concentration risk as documented, computable proxy indicators.
<code>location_score.py</code>	8.8	7-component weighted composite, documented weights, renormalizes when a component is missing.
<code>geo_engine.py</code>	8.9	Top-level <code>GeoResult</code> orchestrator — assembles all of the above, computes nothing itself.
<code>batch_runner.py</code>	—	Runs the full 90-locality portfolio (no sampling needed at this scale).
<code>export_results_csv.py</code>	—	4 numbered CSVs → <code>docs/geo_results_csv/</code> .
<code>generate_charts.py</code>	—	4 charts → <code>docs/geo_charts/</code> .
<code>test_phase8.py</code>	—	26 automated tests (all passing).

5. SECTION 8.7 IN PRACTICE — WHAT WAS NOT INVENTED

This dataset contains **no flood, environmental, or climate/hazard data of any kind** — no elevation, no floodplain boundary, no environmental monitoring reading anywhere in Phase 1.

`geo_risk.compute_geo_risk()` hard-codes `flood_risk` and `environmental_risk` to INSUFFICIENT DATA for **all 90 of 90 localities**, with no exception — verified by a dedicated test

(`test_flood_and_environmental_risk_always_insufficient_data`) that checks this across a spread of locality IDs, not just one.

Two *different* risk dimensions genuinely are computable from real data and are surfaced as explicitly-labeled `PROXY_INDICATOR` values (not calibrated probabilities, same honesty standard Phase 7 set for Monte Carlo):

- **Infrastructure delivery risk** — what share of a locality’s infrastructure impact score is still *planned* (future `expected_completion`) rather than *completed*. Portfolio mean: 57.7/100, median 64.75/100 — meaning a majority of this dataset’s infrastructure story for many localities still depends on projects not yet built.
- **Infrastructure concentration risk** — a Herfindahl-Hirschman-style index over each locality’s

infrastructure impact scores, flagging single-point-of-failure exposure. With only **2.32 infrastructure records per locality on average** (209 items ÷ 90 localities), concentration risk is bimodal: median 4.7/100 (localities with several roughly-even items) but 75th percentile already at 100/100 (localities effectively dependent on one dominant item).

6. A REAL NORMALIZATION BUG FOUND AND FIXED DURING BUILD

The first version of `location_score._market_strength_score()` capped the demand/supply ratio at 1.0 (→100) and absorption_rate at 1.0 (→100), assuming ratio=1.0 meant “balanced, full score.” Checking against the actual data showed this was wrong: pooling all `market_monthly` rows across all localities and months, the overall median demand/supply ratio is **1.64** and median absorption_rate is **1.34** — both already above the assumed ceiling (the per-locality panel definition actually used for the P5/P95 anchors below gives a similar picture: median ratio 1.72, median absorption 2.14, independently re-verified during audit). Either way the conclusion holds: the naive formula saturated **70 of 90 localities (78%) at a market-strength score of exactly 100**, destroying the metric’s ability to differentiate anything.

Fix: computed the actual P5/P95 of each metric’s “latest value per locality” across the full 90-locality dataset (demand/supply ratio: P5=1.06, P95=4.89; absorption_rate: P5=0.48, P95=6.21) and min-max scaled against those real, data-driven anchors instead of an assumed ceiling. After the fix: 0 localities saturated at 100, median score 28.5, full spread 0–90.6. A regression test (`test_market_strength_not_saturated_across_portfolio`) locks this in — asserts fewer than 10% of localities can sit at the ceiling, so this specific bug can’t silently return.

Post-delivery audit note: an independent review re-derived these P5/P95 anchors directly from `data/processed/market_monthly.csv` using the exact definition `location_score.py` uses (12-month-mean demand/supply ratio and latest absorption_rate, per locality) and got a close match (ratio P5/P95 = 1.06/4.79 vs. the code’s 1.06/4.89; absorption P5/P95 = 0.48/6.21 vs. 0.48/6.21) — confirming the anchors are genuinely data-derived, not invented. The only issue found was the narrative median figures quoted above, which had been computed with a simpler pooled-rows aggregation rather than the per-locality-panel definition the code actually uses; the text above has been corrected to show both and no code change was needed. Full portfolio location-score statistics (min 37.6 / median 53.1 / mean 53.6 / max 70.5) and the 31/90 PASS vs. 59/90 PASS WITH LIMITATIONS split were independently re-run end-to-end and matched exactly.

7. FULL-PORTFOLIO RESULTS (n=90 localities — all of them, no sampling)

Metric	Value
<code>validation_status</code> = PASS (all 7 location-score components available)	31 / 90 (34.4%)
<code>validation_status</code> = PASS WITH LIMITATIONS (missing transit and/or commercial infra records)	59 / 90 (65.6%)
Location Score — min / median / mean / max	37.6 / 53.1 / 53.6 / 70.5
Localities with zero infrastructure records	0 / 90
Mean infrastructure records per locality	2.32
<code>flood_risk</code> / <code>environmental_risk</code> availability	0 / 90 (INSUFFICIENT DATA for all, by design)

By city, mean Location Score ranges narrowly from 52.6 (Mumbai) to 54.9 (Bengaluru) — see `docs/geo_charts/location_score_by_city.png`. This narrow spread is itself a finding: this dataset’s locality-level scores don’t show strong city-level clustering, so a caller looking for “which city is best” should not expect a strong signal from this composite alone — the more informative variation is locality-to-locality (min 37.6 to max 70.5), not city-to-city.

8. TESTS EXECUTED / PASSED / FAILED

26/26 passed. Coverage: unknown-locality handling, OBSERVED-tagging correctness, planned/completed count consistency, accessibility correctly returning INSUFFICIENT DATA with zero infra items and PASS WITH LIMITATIONS with partial categories, accessibility score never exceeding 100, closer infrastructure scoring higher than farther (monotonicity), flood/environmental risk INSUFFICIENT DATA across a spread of localities (not just one), delivery-risk boundary cases (0% and 100% planned), concentration-risk boundary cases (single item = maximum, even split = low, uneven split higher than even), the market-strength saturation regression test described above, weights summing to 1.0, renormalization when a component is missing, and end-to-end engine behavior for both a resolvable and an unknown locality.

9. MASTER PROMPT REQUIREMENTS COVERAGE

Section	Requirement	Status
8.3	Locality growth, demand, supply, liquidity, price, market activity	✓ <code>data_loader._market_intelligence</code> , feeds <code>location_score</code>
8.4	Accessibility (road connectivity, transport, distance)	✓ <code>accessibility.compute_accessibility</code>
8.5	Infrastructure (schools, hospitals, transit, commercial)	✓ <code>accessibility.compute_infrastructure_intelligence</code>
8.6	Neighborhood quality (amenities, demand indicators)	✓ <code>accessibility.compute_neighborhood_quality</code>
8.7	Geo risk, never invent	✓ <code>geo_risk.py</code> , flood/environmental correctly INSUFFICIENT DATA everywhere
8.8	Location score with documented components/weights/normalization/limitations	✓ <code>location_score.py</code>
8.9	GeoResult (metrics, locality intel, accessibility, infra, geo risk, score, evidence, limitations)	✓ <code>geo_engine.run_geo_engine</code>
3.2	Never invent	✓ enforced throughout, notably §8.7
13	OBSERVED / DERIVED / PROXY_INDICATOR labeling	✓ every sub-result tagged

10. BUSINESS VALUE

- Every locality now has a transparent, auditable Location Score built entirely from real distances and real pre-validated quality scores — no black-box.
- The accessibility/infrastructure breakdown lets a caller see *why* a locality scores the way it does (e.g., “no transit infrastructure recorded nearby”), not just a single number.
- The infrastructure delivery-risk and concentration-risk proxies surface a genuinely useful, previously invisible signal: a locality can look good on paper (high `infrastructure_score` from Phase 1) while depending heavily on a single not-yet-built project — a caller relying only on the pre-computed `infrastructure_score` would miss this.
- Correctly refusing to fabricate flood/environmental risk protects Phase 9’s Decision Engine from a false sense of completeness on geo-hazard exposure — the `INSUFFICIENT DATA` status is itself the decision-relevant signal (§9.7’s geo-sufficiency gate should read this directly).

11. KNOWN LIMITATIONS (see also `docs/limitations.md`)

1. **No flood, environmental, or hazard data exists anywhere in this platform** — `flood_risk` and `environmental_risk` are `INSUFFICIENT DATA` for all 90 localities, unconditionally. This is a genuine

data gap, not an engine limitation.

2. **Sparse infrastructure coverage:** only 2.32 records per locality on average; 65.6% of localities are missing at least one infrastructure category (transit or commercial), producing `PASS WITH LIMITATIONS`.
3. **Location Score weights (§8.8) are a documented business-judgement default**, not fitted to any real investment-outcome data — no such outcome data exists in this platform to fit against.
4. **Accessibility's distance-decay formula** ($1/(1+\text{distance_km})$) is a documented, simple, monotonic choice — not fitted to real travel-time or transit-frequency data, which doesn't exist in this dataset.
5. **Infrastructure delivery-risk and concentration-risk are proxy indicators**, not calibrated probabilities of any specific event (a project slipping, a locality losing access) — same honesty standard as Phase 7's Monte Carlo UNCALIBRATED status.
6. **expected_completion dates are taken at face value** from `core.infrastructure` — no independent verification that a “completed” project was actually delivered on schedule or as specified.
7. No live PostgreSQL was available in this sandbox; run with `--source csv` (documented dev/test path). Recommend re-running with `--source db` before Phase 9 integration.

12. DEPENDENCIES

Phase 1 (`core.localities`, `core.cities`, `core.infrastructure`, `core.market_monthly`). Independent of Phases 6/7 — can run in parallel, per the master architecture diagram.

13. SECURITY CONSIDERATIONS

Unchanged pattern from Phase 3/5/6/7: no hardcoded credentials, parameterized SQL, `DB_USER` / `DB_PASSWORD` required with no defaults.

14. PERFORMANCE CONSIDERATIONS

90 localities × full pipeline (data load + 5 sub-modules) completes in under 2 seconds in this sandbox. This is the smallest-scale phase in the platform (90 rows vs. 8,000 properties), so no sampling or performance optimization was needed.

15. PRODUCTION READINESS

Per Section 24: **Production-Oriented Prototype**. Locality intelligence, accessibility, infrastructure, geo risk, and location scoring are complete and tested against the real dataset. Recommended before Phase 9 integration: (a) re-run with `--source db` against a live PostgreSQL instance; (b) if the business has access to a real flood/environmental hazard dataset for these cities, wire it into `data_loader.py` and `geo_risk.py` — the `INSUFFICIENT DATA` placeholders are designed to be replaced by an `OBSERVED` source, not by an invented estimate; (c) have the business review the Location Score's default equal-ish weights (§8.8) and confirm or adjust them.

16. NEXT PHASE DEPENDENCIES

Phase 9's Geo Agent should call `geo_engine.run_geo_engine()` directly. Its `geo_sufficiency` input to the Decision Engine's gate (§9.7) should read `validation_status` and the `flood_risk/environmental_risk` `INSUFFICIENT DATA` status directly from this phase's output — a locality with `PASS WITH LIMITATIONS` or missing hazard data should reduce decision confidence, not be silently treated as fully covered.

17. FINAL STATUS

PHASE 8: COMPLETE. Locality intelligence, accessibility, infrastructure intelligence, neighborhood quality, geo risk, and a documented Location Score composite all built and tested against the real 90-locality/209-infrastructure-record dataset — 26/26 automated tests passing, flood/environmental risk correctly never invented, and one real normalization bug found and fixed during build (market-strength scoring saturated 78% of localities before the fix). Ready for Phase 9 integration alongside Phases 6/7.